Math 118: Mathematical Methods of Data Theory

Lecture 25: Neural Networks, specifically Backpropagation

Instructor: Daniel Mckenzie

Dept. of Mathematics, UCLA

March 11, 2020

Logistic Regression, again

Recall the following facts about logistic regression:

- **Sigmoid:** $\sigma(z) = \dfrac{1}{1+e^{-z}}$
- **Recall:** $h_{\boldsymbol{\theta},b}(\boldsymbol{x}) = \sigma(\boldsymbol{\theta}^\top \boldsymbol{x} + b)$ where , $\boldsymbol{\theta} \in \mathbb{R}^{\mathbf{d}}$.
- $\ell_i(\boldsymbol{\theta}, b) = -y^{(i)} \log(h_{\boldsymbol{\theta},b}(\boldsymbol{x}^{(i)}) - \left(1 - y^{(i)}\right) \log\left(1 - h_{\boldsymbol{\theta},b}(\boldsymbol{x}^{(i)})\right)$.
- $L(\boldsymbol{\theta}, b) = \sum_{i=1}^{N} \ell_i(\boldsymbol{\theta}, b)$.
- Find $\boldsymbol{\theta}^*, b^* = \arg\min_{\boldsymbol{\theta}\in\mathbb{R}^d, b\in\mathbb{R}} L(\boldsymbol{\theta}, b)$ using gradient descent:

$$(\theta_i)_{k+1} = (\theta_i)_k - \alpha \frac{\partial L}{\partial \theta_i}$$

$$b_{k+1} = b_k - \alpha \frac{\partial L}{\partial b}$$

Logistic Regression, with better notation

- Define $\tilde{\boldsymbol{x}} = [\boldsymbol{x}^\top, 1]^\top \in \mathbb{R}^{d+1}$.
- Define $\tilde{\boldsymbol{\theta}} = [\boldsymbol{\theta}^\top, b] \in \mathbb{R}^{d+1}$.
- **Then:** $h_{\boldsymbol{\theta},b}(\boldsymbol{x}) = \sigma(\boldsymbol{\theta}^\top \boldsymbol{x} + b) = \sigma(\tilde{\boldsymbol{\theta}}^\top \tilde{\boldsymbol{x}}) =: h_{\tilde{\boldsymbol{\theta}}}(\tilde{\boldsymbol{x}})$.
- Define the **log-loss** function: $\eta_i(z) = -y^{(i)} \log(z) - (1 - y^{(i)}) \log(1 - z)$. Observe that:

$$\ell_i(\boldsymbol{\theta}, b) = \eta_i(h_{\tilde{\boldsymbol{\theta}}}(\tilde{\boldsymbol{x}}^{(i)}) = \eta_i\left(\sigma(\tilde{\boldsymbol{\theta}}^\top \tilde{\boldsymbol{x}}^{(i)})\right) := \tilde{\ell}_i(\tilde{\boldsymbol{\theta}})$$

- Why are we doing this? Makes chain rule easier:

$$\begin{aligned}
\frac{\partial \tilde{\ell}_i}{\partial \tilde{\theta}_j} &= \frac{\partial \eta_i(z)}{\partial z}\Big|_{z=\sigma(\tilde{\boldsymbol{\theta}}^\top \tilde{\boldsymbol{x}}^{(i)})} \frac{\partial \sigma(a)}{\partial a}\Big|_{a=\tilde{\boldsymbol{\theta}}^\top \tilde{\boldsymbol{x}}^{(i)}} \frac{\partial}{\partial \tilde{\theta}_j}\left[\tilde{\boldsymbol{\theta}}^\top \tilde{\boldsymbol{x}}^{(i)}\right] \\
&= \frac{\partial \eta_i(z)}{\partial z}\Big|_{z=\sigma(\tilde{\boldsymbol{\theta}}^\top \tilde{\boldsymbol{x}}^{(i)})} \frac{\partial \sigma(a)}{\partial a}\Big|_{a=\tilde{\boldsymbol{\theta}}^\top \tilde{\boldsymbol{x}}^{(i)}} \tilde{x}_j^{(i)}
\end{aligned}$$

Features

- $x_1, \ldots, x_d$ are called our **features**.
- Key drawback of logistic regression: decision boundary must be a straight line in $\mathbb{R}^d_{x_1,\ldots,x_d}$:

$$\boldsymbol{\theta}^\top \boldsymbol{x} + \theta_{d+1} = 0$$

- **Idea:** what if we make better features?

$$z_1 = f_1(x_1, x_2, \ldots, x_d)$$

$$\ldots$$

$$z_d = f_d(x_1, x_2, \ldots, x_d)$$

- Many ways to choose f_i. Important that the f_i are **non-linear**. Why?
- Suppose $z_i = \sum_{j=1}^d A_{ij} x_j$, then $\boldsymbol{z} = A\boldsymbol{x}$ and so $\boldsymbol{x} = A^{-1}\boldsymbol{z}$.
- Decision boundary in $\mathbb{R}^d_{z_1,\ldots,z_d}$:

$$\boldsymbol{\theta}^\top \left(A^{-1}\boldsymbol{z}\right) + \theta_{d+1} = 0 \Rightarrow \left(\boldsymbol{\theta}^{\text{new}}\right)^\top \boldsymbol{z} + \theta_{d+1} = 0 \quad \text{where } \boldsymbol{\theta}^{\text{new}} = \left(A^{-1}\right)^\top \boldsymbol{\theta}$$

Feature Learning

- **Key (modern) idea of ML:** Choose f_i to be simplest possible non-linear function of $x_1, \ldots, x_d$:

$$z_i = \tau\left(\sum_{j=1}^{d} W_{ij} x_j\right)$$

- Choose $\tau(z) : \mathbb{R} \to \mathbb{R}$ to be a simple, **scalar nonlinearity** *e.g.* $\tau(z) = \sigma(z)$.
- **Learn** W_{ij} from data.
- Some remarks:
  - Don't need to have same number of z_i as x_j. Can have $x_1, \ldots, x_{d_1}$ and $z_1, \ldots, z_{d_2}$.
  - Note $W \in \mathbb{R}^{d_2 \times d_1}$. Often write:

$$\boldsymbol{z} = \tau(W\boldsymbol{x}) \quad \text{Implicitly: Apply } \tau \text{ compentwise}$$

  - Can iterate this idea: $\boldsymbol{w} = \tau(U\boldsymbol{z})$. Better notation:

$$\boldsymbol{z}^{(2)} = \tau\left(W^{(2)}\boldsymbol{z}^{(1)}\right) = \tau\left(W^{(2)}\tau\left(W^{(1)}\boldsymbol{x}\right)\right) = \tau \circ W^{(2)} \circ \tau \circ W^{(1)}(\boldsymbol{x})$$

Neural Networks

- **Definition:** a Neural Network for classification is any function $\boldsymbol{z}^{(k)} = f(\boldsymbol{x}; W^{(1)}, \ldots, W^{(k)})$ where:
  - $f(\boldsymbol{x}; W^{(1)}, \ldots, W^{(k)}) = \tau \circ W^{(k)} \circ \tau \circ W^{(k-1)} \circ \ldots \circ \tau \circ W^{(1)}(\boldsymbol{x})$
  - Parameters $W^{(m)} \in \mathbb{R}^{d_m \times d_{m-1}}$.
  - $\boldsymbol{z}^{(k)} \in \mathbb{R}^{d_k}$ and $d_k = \#$ categories -1. Interpretation:

$$\mathbb{P}[\boldsymbol{x} \text{ in category } i] = z_i^{(k)} \quad \text{for } i = 1, \ldots, d_k$$

$$\mathbb{P}[\boldsymbol{x} \text{ in category } d_k + 1] = 1 - \sum_{i=1}^{d_k} z_i^{(k)}$$

Training Neural Networks

We restrict our attention to 2 category case, so $d_k = 1$.

- Given labeled data $\left\{(\boldsymbol{x}^{(1)}, y^{(1)}), \ldots, (\boldsymbol{x}^{(N)}, y^{(N)})\right\}$ where $y^{(i)} = 0$ or 1.
- Choose a loss function. Examples:

$$\ell_i(W) = \left(f(\boldsymbol{x}^{(i)}; W^{(1)}, \ldots, W^{(k)}) - y^{(i)}\right)^2$$
$$\ell_i(W) = \eta_i(f(\boldsymbol{x}^{(i)}; W^{(1)}, \ldots, W^{(k)})$$
$$\text{where } \eta_i(z) = -y^{(i)} \log(z) - (1 - y^{(i)}) \log(1 - z)$$

- Define **Total Loss:** $L(W) = \sum_{i=1}^{N} \ell_i(W)$
- Find $W^* = \arg\min L(W)$ using gradient descent.
- **Problem:** GD only converges to local minima. When $L(W)$ is not convex no guarantee that GD will find W^*, but it usually find a "good enough" W.

Backpropagation

- Consider 2 layers and 2 categories:

$$f(\boldsymbol{x}; W^{(1)}, W^{(2)}) = \sigma(W^{(2)}\sigma(W^{(1)}\boldsymbol{x}))$$

- Gradient descent iteration:

$$\left(W^{(m)}_{j_1,j_2}\right)_{t+1} = \left(W^{(m)}_{j_1,j_2}\right)_{t} - \alpha\frac{\partial L((W)_t)}{\partial W^{(m)}_{j_1,j_2}}$$

- Use $\ell_i(W) = \eta_i(f(\boldsymbol{x}^{(i)}; W^{(1)}, \ldots, W^{(k)})$ and $L(W) = \sum_{i=1}^{N} \ell_i(W)$

- Note: $\dfrac{\partial L(W)}{\partial W^{(m)}_{j_1,j_2}} = \sum_{i=1}^{N} \dfrac{\partial \ell_i(W)}{\partial W^{(m)}_{j_1,j_2}}$ so just need to determine $\dfrac{\partial \ell_i(W)}{\partial W^{(m)}_{j_1,j_2}}$.

Backpropagation

- Observe $\ell_i(W) = \eta_i\left(\sigma(W^{(2)}\sigma(W^{(1)}\boldsymbol{x}))\right)$.
- Will determine $\dfrac{\partial \ell_i(W)}{\partial W^{(m)}_{j_1,j_2}}$ using chain rule.
- Useful notation:

$$\boldsymbol{z}^{(0)} = \boldsymbol{x} \quad \text{and } \boldsymbol{a}^{(1)} = W^{(1)}\boldsymbol{x}$$
$$\boldsymbol{z}^{(1)} = \sigma(\boldsymbol{a}^{(1)}) \quad \text{and } \boldsymbol{a}^{(2)} = W^{(2)}\boldsymbol{z}^{(1)} \text{ and } \boldsymbol{z}^{(2)} = \sigma(\boldsymbol{a}^{(2)})$$
$$\text{so } \ell_i(W) = \eta_i(\boldsymbol{z}^{(2)})$$

- Chain Rule: $\dfrac{\partial \ell_i(W)}{\partial W^{(2)}_{j_1,j_2}} = \displaystyle\sum_{r=1}^{d_2} \frac{\partial \eta_i}{\partial z^{(2)}_r}\frac{\partial z^{(2)}_r}{\partial W^{(2)}_{j_1,j_2}} = \frac{\partial \eta_i}{\partial z^{(2)}_{j_1}}\frac{\partial z^{(2)}_{j_1}}{\partial W^{(2)}_{j_1,j_2}}$

$$\frac{\partial \ell_i(W)}{\partial W^{(2)}_{j_1,j_2}} = \frac{\partial \eta_i}{\partial z^{(2)}_{j_1}}\frac{\partial z^{(2)}_{j_1}}{\partial \boldsymbol{a}^{(2)}}\frac{\partial \boldsymbol{a}^{(2)}}{\partial W^{(2)}_{j_1,j_2}} = \frac{\partial \eta_i}{\partial z^{(2)}}\sum_{i=1}^{d_1}\frac{\partial z^{(2)}_{j_1}}{\partial a^{(2)}_i}\frac{\partial a^{(2)}_i}{\partial W^{(2)}_{j_1,j_2}}$$
$$= \left(-\frac{y^{(i)}}{z^{(2)}} + \frac{1-y^{(i)}}{1-z^{(2)}}\right)\left(\frac{\partial z^{(2)}_{j_1}}{\partial a^{(2)}_{j_1}}\frac{\partial a^{(2)}_{j_1}}{\partial W^{(2)}_{j_1,j_2}}\right)$$

- Useful notation:

$$\boldsymbol{z}^{(0)} = \boldsymbol{x} \quad \text{and } \boldsymbol{a}^{(1)} = W^{(1)}\boldsymbol{x}$$
$$\boldsymbol{z}^{(1)} = \sigma(\boldsymbol{a}^{(1)}) \quad \text{and } \boldsymbol{a}^{(2)} = W^{(2)}\boldsymbol{z}^{(1)} \text{ and } \boldsymbol{z}^{(2)} = \sigma(\boldsymbol{a}^{(2)})$$
$$\text{so } \ell_i(W) = \eta_i(\boldsymbol{z}^{(2)})$$

- So far: $\dfrac{\partial \ell_i(W)}{\partial W^{(2)}_{j_1,j_2}} = \left(-\dfrac{y^{(i)}}{z^{(2)}} + \dfrac{1-y^{(i)}}{1-z^{(2)}}\right)\left(\dfrac{\partial z^{(2)}_{j_1}}{\partial a^{(2)}_{j_1}}\dfrac{\partial a^{(2)}_{j_1}}{\partial W^{(2)}_{j_1,j_2}}\right)$.
- Now: $\dfrac{\partial z^{(2)}_{j_1}}{\partial a^{(2)}_{j_1}} = \dfrac{\partial}{\partial a^{(2)}_{j_1}}\left[\sigma(a^{(2)}_{j_1})\right] = \sigma(a^{(2)}_{j_1})\left(1-\sigma(a^{(2)}_{j_1})\right) = z^{(2)}_{j_1}\left(1-z^{(2)}_{j_1}\right)$
- And: $\dfrac{\partial a^{(2)}_{j_1}}{\partial W^{(2)}_{j_1,j_2}} = z^{(1)}_{j_2}$
- So: $\dfrac{\partial \ell_i(W)}{\partial W^{(2)}_{j_1,j_2}} = \left(-\dfrac{y^{(i)}}{z^{(2)}} + \dfrac{1-y^{(i)}}{1-z^{(2)}}\right)\sigma(a^{(2)}_{j_1})\left(z^{(2)}_{j_1}\left(1-z^{(2)}_{j_1}\right)\right)z^{(1)}_{j_2}$.
- **Exercise:** Derive similar formula for $\dfrac{\partial \ell_i(W)}{\partial W^{(1)}_{j_1,j_2}}$

So, the $t+1$-st step of Gradient Descent for NN training looks like:

- Do a **forward pass** through the network. Compute and store for all i:

$$\begin{aligned}
&\boldsymbol{z}^{(0)} = \boldsymbol{x}^{(i)} \quad \text{and } \boldsymbol{a}^{(1)} = W^{(1)}\boldsymbol{x} \\
&\boldsymbol{z}^{(1)} = \sigma(\boldsymbol{a}^{(1)}) \quad \text{and } \boldsymbol{a}^{(2)} = W^{(2)}\boldsymbol{z}^{(1)} \text{ and } \boldsymbol{z}^{(2)} = \sigma(\boldsymbol{a}^{(2)}) \\
&\text{so } \ell_i(W) = \eta_i(\boldsymbol{z}^{(2)})
\end{aligned}$$

  Note here $W^{(m)} = (W^{(m)})_t$, but we omit the t subscript to make the notation more palatable.

- Do a **backward pass** through the network. Compute:

$$\frac{\partial \ell_i(W)}{\partial W^{(2)}_{j_1,j_2}} \text{ and } \frac{\partial \ell_i(W)}{\partial W^{(1)}_{j_1,j_2}}$$

  using stored values.

- **Update parameters:**

$$W^{(m)}_{j_1,j_2} = W^{(m)}_{j_1,j_2} - \alpha \sum_{i=1}^{N} \frac{\partial \ell_i(W)}{\partial W^{(m)}_{j_1,j_2}} \quad \text{for all } m, j_1 \text{ and } j_2$$